

ClipSync

Real-Time Cross-Device Clipboard Synchronization

Project Documentation

Full-Stack Web Application | React + Node.js + MongoDB + Socket.IO

Author Nitesh	Type Personal Project	Version 1.0
-------------------------	---------------------------------	-----------------------

1. Project Overview

ClipSync is a real-time, cross-device clipboard synchronization web application that allows users to instantly share text snippets across all their signed-in devices. Once a user authenticates via Google OAuth, any text they submit from one device is immediately pushed to all their other active sessions — making it effortless to move content between a laptop, phone, and tablet without resorting to email or messaging apps.

Core Value Proposition

Stop emailing yourself. Stop texting yourself. Type it once on any device, and it appears everywhere you are signed in — instantly.

1.1 The Problem It Solves

Moving small pieces of text between devices is a surprisingly painful experience in everyday workflows: copying a link on mobile to use on desktop, transferring a code snippet from a laptop to another machine, or sharing a quick note across devices. Existing solutions either require installing native apps, are restricted to Apple/Windows ecosystems, or lack real-time push behavior. ClipSync is browser-based, cross-platform, and instant.

1.2 Key Features

- Real-time push via WebSockets — no polling, no refresh needed
- Google OAuth login — no passwords to manage, identity is always verified
- Strict per-user data isolation — your clips are only visible to your sessions
- Automatic 6-hour clip expiry — data does not accumulate indefinitely
- Visual clip cards with expiry countdown timers and delete controls
- NEW badge and glow animation for freshly received clips on other devices
- Random color-coded cards with consecutive duplicate prevention for visual clarity

2. System Architecture

ClipSync follows a clean client-server architecture. The React frontend handles all UI and socket communication, while the Node.js/Express backend manages authentication, database persistence, and real-time broadcast logic.

2.1 High-Level Data Flow

Clip Submission Flow

User types text → POST /clips (JWT auth) → MongoDB saves clip with TTL index → Socket.IO broadcasts clip:new to user room → All connected devices receive and render the clip in real time.

Authentication Flow

User clicks Sign In → Google OAuth redirect → Passport.js validates token → Backend issues JWT → Frontend stores JWT → All subsequent API and socket connections carry the token for identity verification.

2.2 Real-Time Sync Design

Each authenticated user is assigned a private Socket.IO room keyed by their `userId` (`user:{userId}`). When a clip is submitted, the server broadcasts to this room, which covers all active device sessions for that user. This design provides complete isolation without any per-event authorization overhead — room membership itself acts as the auth gate.

2.3 Data Expiry Strategy

Clip expiry is handled natively by a MongoDB TTL index on the `createdAt` field, set to 21,600 seconds (6 hours). This offloads expiry entirely to the database engine — no cron jobs, no scheduled tasks, and no risk of memory leaks from application-level timers. The TTL index purge runs approximately every 60 seconds on MongoDB, which is well within acceptable precision for this use case.

3. Technology Stack

The stack was chosen for full-stack JavaScript consistency, minimal operational overhead, and native support for the real-time communication patterns the app requires.

Layer	Technology	Purpose
Frontend	React (Vite)	Component-based UI with fast HMR dev experience
Frontend	Socket.IO Client	WebSocket connection management and event handling
Backend	Node.js / Express	REST API server, middleware, and route handling

Backend	Socket.IO Server	WebSocket server, room management, and event broadcast
Auth	Passport.js	Google OAuth 2.0 strategy and session scaffolding
Auth	JSON Web Tokens	Stateless auth tokens for API and socket verification
Database	MongoDB	Document store for clips with native TTL index support
ODM	Mongoose	Schema definition, TTL config, and query interface
Build	Vite	Frontend bundler with fast cold starts and HMR
Runtime	Node.js	JavaScript runtime for the backend server

4. Project Structure

4.1 Frontend

The frontend is a Vite-powered React application. Key files and responsibilities:

- `useClipSync.jsx` / `useClips.jsx` — Custom hooks that own the Socket.IO lifecycle, event listeners (`clips:init`, `clip:new`, `clip:removed`), and sync state
- `Input.jsx` — Text input component with live WebSocket connection status badge and Enter-to-send behavior
- `Home.jsx` — Main view rendering clip cards, expiry countdown timers, delete buttons, and the empty-state UI
- `getRandomStyle()` — Color assignment utility with a `do...while` loop preventing consecutive duplicate card colors

4.2 Backend

The backend is an Express application with Socket.IO mounted on the same HTTP server instance. Key components:

- Clip model — Mongoose schema with a TTL index (`expireAfterSeconds: 21600`) on `createdAt`
- Socket middleware — JWT verification on socket handshake before room assignment
- Socket rooms — `user:{userId}` rooms for per-user broadcast isolation
- REST API — GET, POST, DELETE `/clips` routes with JWT-enforced ownership checks

4.3 Environment Variables

Variable	Side	Description
<code>VITE_SERVER_URL</code>	Frontend	Backend server URL for API calls and Socket.IO connection
<code>JWT_SECRET</code>	Backend	Secret key used to sign and verify JSON Web Tokens
<code>MONGO_URI</code>	Backend	MongoDB connection string (Atlas or local)
<code>SESSION_SECRET</code>	Backend	Secret for Express session used during OAuth flow
<code>FRONTEND_URL</code>	Backend	Allowed CORS origin for the React frontend

5. Key Technical Learnings

5.1 WebSockets vs. Polling

Building ClipSync made the cost of HTTP polling concrete. A naively polled version would hammer the server every few seconds per device — wasteful and still delivering stale data in the gaps. Socket.IO's persistent TCP connection flips this: the server pushes only when there is actually something new. Latency drops from polling interval to near-zero, and server load scales with event frequency rather than device count.

5.2 MongoDB TTL Indexes as a Cron Replacement

The 6-hour auto-expiry requirement could have been solved with a cron job querying and deleting old documents on a schedule. TTL indexes are strictly better for this use case: the logic lives at the database layer, it survives application restarts with no state to manage, and there is no risk of a missed run causing data to accumulate. The trade-off is that MongoDB's TTL sweep runs on a roughly 60-second interval, so the exact deletion time can drift — acceptable for a clipboard app, but worth noting for stricter SLAs.

5.3 JWT for Stateless Authentication

Using JWTs for both REST and WebSocket authentication removed the need for session lookups on every request. Once the token is signed and issued post-OAuth, the server can verify identity purely from the token payload on each request — no database round-trips for auth. The socket middleware validates the JWT on connection handshake, so room assignment only happens for verified users.

5.4 Socket Rooms as an Isolation Primitive

Rather than filtering broadcasts at the application level (e.g., checking `userId` on every emitted event), using `Socket.IO` rooms tied to `userId` pushes isolation into the transport layer. A clip broadcast to room `user:abc123` is never visible to a socket in room `user:xyz789` — not as a filtering step, but as a structural guarantee. This pattern scales cleanly: adding a new device just means joining the correct room on connection.

5.5 Optimistic UI with Server Confirmation

On the sending device, the clip is rendered immediately on submit without waiting for the server round-trip. The server then broadcasts the saved clip to the full user room — which includes the sender — meaning remote devices also receive the canonical, database-persisted version. This creates a smooth experience: the sender sees instant feedback, and remote devices receive an event-driven push.

5.6 Preventing Consecutive Duplicate Colors

A pure `Math.random()` color assignment would occasionally produce visually jarring consecutive cards of the same color. A `do...while` loop tracking the last assigned color index resolves this with minimal code: the loop re-rolls until it lands on a different index, guaranteeing visual variety without any complex state management.

6. Real-World Use Cases

6.1 Developer Workflows

- Copy a GitHub URL, API key, or terminal command from desktop to mobile without opening a messaging app
- Transfer a code snippet from one machine to another during a debugging session
- Share a connection string or environment value across dev machines quickly

6.2 Everyday Productivity

- Copy a meeting link from laptop to phone before heading to a conference room
- Send yourself a phone number, address, or note looked up on one device for use on another
- Transfer article links, quotes, or research snippets across devices mid-workflow

6.3 Why It Beats the Alternatives

Texting yourself is asynchronous and creates clutter. Email has overhead and is not designed for ephemeral snippets. Apple Universal Clipboard and Windows Clipboard History are platform-locked. Cloud-based clipboard managers require installing native apps and granting system-

level permissions. ClipSync is browser-native, cross-platform, requires only a Google login, and auto-cleans after 6 hours.

7. Potential Improvements & Next Steps

7.1 Feature Extensions

- Rich content support: images, files, and formatted text beyond plain strings
- Clip categories or tags for organizing frequently reused snippets
- Pin important clips to exempt them from the 6-hour TTL
- One-tap copy button on clip cards to avoid manual selection
- Browser extension for direct clipboard capture without visiting the web app

7.2 Technical Improvements

- End-to-end encryption: encrypt clip content client-side before sending so the server never sees plaintext
- Redis Pub/Sub for horizontal scaling — currently Socket.IO rooms are in-memory, making multi-instance deployment non-trivial
- Refresh token rotation to replace the current single JWT issuance flow
- Rate limiting on clip submission to prevent abuse
- Socket reconnection handling with re-sync of missed clips during disconnection windows

7.3 Production Readiness

- Containerize with Docker for consistent deployments
- Add CI/CD pipeline with automated testing before deploy
- Observability: structured logging, error tracking, and uptime monitoring
- PWA manifest for home screen installation on mobile

8. Summary

What Was Built

A production-pattern full-stack web application combining Google OAuth, JWT-secured REST APIs, Socket.IO real-time broadcasting, MongoDB with TTL-indexed auto-expiry, and a React frontend with custom hooks encapsulating sync logic. Every layer has a deliberate technical justification.

What Was Learned

WebSocket room design for multi-device isolation, TTL indexes as a zero-maintenance expiry primitive, stateless JWT authentication across REST and WebSocket transports, optimistic UI patterns, and the value of encapsulating transport logic in custom React hooks.

What It Demonstrates

ClipSync is a compact but technically complete project that touches authentication, real-time infrastructure, database design, and frontend state management simultaneously — proving that all these layers can be integrated cleanly without over-engineering.